

Document number: P2858R0
Date: 2023-05-12
Audience: SG21
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>

Noexcept vs contract violations

The purpose of this document is to highlight some design issues concerning `noexcept` functions with preconditions or postconditions or any contract checks evaluated inside their bodies. [\[P2388R4\]](#) proposed only hard `std::abort()` upon detecting a contract violation, therefore it didn't have to address these questions. Now that we are considering extensions like *Eval_or_throw* ([\[P2698R0\]](#)) or contract violation handlers ([\[P2811R2\]](#)), the interaction with `noexcept` has to be designed.

The meaning of noexcept

The original motivation for adding `noexcept` was for users to annotate their move constructors with it, in order that the `vector` implementation could statically detect that property and select the most appropriate algorithm for providing the strong exception safety guarantee. The problem is described in [\[N2855\]](#). In this view, the motivation for putting the `noexcept` specifier is to inform generic components which implementation to apply. It has only loose connection with whether a function throws or not. That is, the specification for function `f` can say that it never throws an exception but `f` doesn't have to be `noexcept`, and conversely: a function can be declared `noexcept` and it can still throw an exception in the function body. Such exception will not leave the function body (due to a call to `std::terminate`) but it is still an undesired situation, detectable only dynamically. The way `noexcept` specifier is consumed is via operator `noexcept`, type traits such as `std::is_nothrow_move_constructible`, or more directly via type system:

```
void fun() noexcept;

void overload(void(*)());           // #1
void overload(void(*)() noexcept); // #2

overload(&fun);                     // #2 selected
```

Another view that people often present is that `noexcept` is a function's postcondition, much like the guarantee that a value produced by function `sqrt` is non-negative. However, this view overlooks one important aspect of a postcondition: a postcondition is a *conditional* guarantee: a function guarantees to satisfy the postcondition only if its preconditions have been satisfied. If the function's precondition is not satisfied, it is not required to satisfy its postcondition. To illustrate this, consider the following correct implementation of function `select_ptr`:

```
int* select_ptr(int* lhs, int* rhs, bool cond)
[[pre: lhs != nullptr]]
[[pre: rhs != nullptr]]
[[post ans: ans != nullptr]]
{
    return cond ? rhs : lhs;
}
```

Clearly, this function can return a null pointer, if it is passed null pointers as arguments, but this is ok as the function is not bound by the contract if its precondition is violated. `noexcept`, on the other hand, is a property that has effect even if the function is called out of contract.

An author of a function can guarantee in the function's documentation that the function never throws when its precondition is satisfied, but they do not have to declare the function as `noexcept`. That is, the lack of `noexcept`

does not imply the lack of no-throw guarantee.

Can different translation modes alter the type system?

Preconditions and postconditions are to be put on function declarations, which means that if their expressions are runtime-evaluated, this evaluation happens somewhere between the caller and the callee. Now, if a contract violation handler — that can potentially throw — is added to the picture, the question arises, how this should affect language constructs that detect the `noexcept`.

```
void fun() noexcept [[pre: false]];

constexpr bool mystery = noexcept(fun()); // what value??

using nothrow_callback_t = void(*)() noexcept;
nothrow_callback_t p = fun;                // compiles??

void overload(void(*)());                  // #1
void overload(void(*)() noexcept);         // #2

overload(&fun);                            // which overload??
```

Should the answer to these questions be different in different translation modes? Should different implementations be allowed to give different answers? This is what [\[P2852R0\]](#) proposes, with the possibility to strengthen this in the future.

No-fail, no-throw and similar guarantees

Some contexts of a program require a no-fail guarantee while some other require a no-throw guarantee, and these two guarantees are different.

A no-fail guarantee is required as a building block for providing a strong — transactional — guarantee. This type of guarantee is often required from `swap` functions. They should be simple enough that there is no way that anything could fail when executing them. Similarly, on the other side of the interface, it is often possible to implement `swap` as a number of raw pointer assignments, so one can easily convince oneself that no fail is possible.

There are other situations where a fail is acceptable, but such failure should not be reported. In case a program reports failures via error codes or things like `std::expected`, the failure can simply be ignored; but in case of exceptions, where the information about failure is propagated by default, it is required that no exception is thrown even in the case of a failure. This is often the case for destructors: when they are used to release resources. Typically, a destructor is called in a function after the return object, if any, has been initialized, and the postcondition of the function has already been satisfied. In that case throwing an exception from the function would be a design error. Releasing a resource can fail, but this does not affect the satisfaction of the postconditions, so breaking the caller is not appropriate.

Obviously, a no-fail guarantee implies the no-throw guarantee, but the implication doesn't work in the other direction.

Apart from these, we also have conditional no-fail guarantees: for ranges of input values. For instance, `vector<T>::at` guarantees no fail provided that the index does not exceed the size of the vector. Similarly, `vector<int>::push_back` guarantees no fail, provided that the vector's capacity is greater than its size. The no-fail guarantee obviously is also predicated on satisfying the function's preconditions. As a consequence, the following is a valid implementation of `vector<T>::operator[]`:

```
constexpr vector<T>::operator[](size_type pos) const
[[pre: pos < this->size()]]
// no-fail
```

```
{
    return this->at(pos);
}
```

When programmers need to provide bodies of *no-fail* functions they can:

- call other no-fail functions,
- call conditional no-fail functions, and make sure that their conditions are met.

When programmers need to provide bodies of *no-throw* functions they can:

- call other no-fail functions,
- call other no-throw functions, provided that a failure in the no-throw function does not affect the correctness of subsequent operations,
- call conditional no-fail functions, and make sure that their conditions are met,
- call any function but catch all exceptions thrown and conceal them.

Naturally, all these guarantees cannot be expressed as attributes or specifiers on the function declaration. They can only be learned from the documentation of some sort. Only documentation can describe the full contract of a given function. If we look at it from the perspective of a user, they can ask, where does a new C++ programmer should take the documentation from for the Standard Library? There is no easy answer to this question. Because of that there is a temptation to use `noexcept` specifier to identify no-throw guarantee, as a substitute for documentation. But this has its problems: the real no-fail guarantee is conditional: it depends on the input values. It is easy to accept for the case of `vector<int>::push_back`, less so for `vector<T>::operator[]`, but the situation is the same. The implementation of `vector<T>::operator[]` can only make guarantees in the case where the index is within a certain range of values. The function shouldn't be forced to make any guarantees for the cases where its precondition isn't satisfied. That is the point of a contract. No-fail, no-throw are also guarantees, so they should be (allowed to be) conditional. This is in contrast with the mechanics of `noexcept` which allows code transformations regardless if functions are called with their precondition satisfied or not.

One could say that if a function is called with its precondition violated then all bets are off and all the above reasoning can be discarded. But this is not entirely correct. Calling a language feature out of contract is UB (Undefined Behavior), calling a Standard Library function out of contract is, at least for now, UB (even if of different kind), but calling user-defined functions out of contract is not necessarily a UB. Consider the above example of `select_ptr`. Its implementation has no UB, not even on illegal inputs. Yet, because it specifies a precondition, it is not bound to offer any guarantee upon a violated precondition.

How about the situation where a function has no preconditions: it has defined semantics for every input value. Can we use `noexcept` on such function to mean no-throw guarantee? First, a precondition is more than just a program state that we can observe and give a true/false answer. A precondition may be "you can call `f` only if `g` hasn't been called" or conversely "you can call `run` only if `init` has been called". These things can still trigger a contract violation somewhere inside the implementation, but the function's input values (or the observable program state) are fine. But there is also the question of behavior in the face of bugs. The caller may have done everything correctly, but the implementation of the function may have a bug that needs to be signaled via the contract violation handler. This is covered in the next section.

Throwing from a `noexcept` function

There is one clever use for a `noexcept` specifier. This is when I need my program to terminate when a given function in a given context ends in failure that is reported with an exception. I can put a `noexcept` there to mean: convert an exception into a call to `std::terminate()`. We will assume that this use case is rare, and skip it from further analysis.

Other than the above, when a function author annotates it as `noexcept`, it means that they commit to providing implementation that *never* ends in exception. Causing the call to `std::terminate` is unintentional, and maybe unacceptable. Suppose we have a function with no precondition that is declared `noexcept`. Providing a no-fail

implementation is known to be possible, but we have a bug in our implementation. If we have a contract support framework in place, bugs in the implementation can be detected via contract declaration checks. Additionally, when a contract check can throw an exception upon any bug in the implementation declared in this way — such as in *Eval_or_throw* mode ([\[P2698R0\]](#)) or a throwing contract violation handler ([\[P2811R2\]](#)) — we will end up throwing an exception from a `noexcept` function, and causing a call to `std::terminate()`. Is this an acceptable solution? One could say that if the program has a bug, you cannot expect anything of it. But on the other hand, one of the purposes of the contract support framework is to make programs behave reliably even in the face of an exception.

[\[P2698R0\]](#) argues that in some environments stopping a program is unacceptable. If we wanted to implement this requirement, we would have to do one of the following:

- disallow throwing from the contract checks,
- disallow annotating functions as `noexcept`,
- drastically minimize/discourage the usage `noexcept` and tolerate the crashes in these situations.

If the third option is preferred, it would have to be accompanied by guidelines for:

- WG21: how to specify the interfaces of the Standard Library functions,
- programmers: how to specify the interfaces of their functions.

It should be noted that `noexcept` is declared implicitly for destructors and deallocation functions and these can also have preconditions, and they can surely have bugs detectable via the contract support framework mechanisms. Consider one example from [\[P2784R0\]](#):

```
class State
{
    std::vector<Column*> _columns;
    unsigned _theColumn;

public:
    bool invariant() const noexcept
    {
        return _theColumn < _columns.size() && _columns[_theColumn] != nullptr;
    }

    void alter()
        [[pre: invariant()]]
        [[post: invariant()]];

    ~State()
        [[pre: invariant()]]
    {
        delete _columns[_theColumn];
    }
};
```

If upon calling `state.alter()` its precondition is violated and this gets turned into an exception, during the stack unwinding we will need to call the destructor of `state`. The destructor also has a precondition which would also be violated. This would trigger a second exception to be thrown.

Apart from implicit `noexcept`, there strong motivation for adding the specifier explicitly is the original motivation from [\[N2855\]](#). Even move constructors can have bugs that will end up throwing exceptions when a throwing contract violation handler is in place.

Special-casing preconditions on library boundaries

[\[P2780R0\]](#) presents an important use case, where a program uses a third-party library. We do not want to check the contract violations in the used library, nor its postconditions: we trust the library. We only want to check contract violations in our program, which includes checking the preconditions of the library's functions where possible (it is unimplementable when these function are called via indirection). While this is the case worth supporting, the discussion about `noexcept` in this paper is an unnecessary distraction. Counter to what the paper suggests, its proposed solution does not address the problem of throwing violation handling mode: neither *Eval_or_throw* mode ([\[P2698R0\]](#)) nor a throwing contract violation handler ([\[P2811R2\]](#)).

Performing the precondition check *always* in the caller — which [\[P2780R0\]](#) essentially proposes — leaves the problems with the type system and type traits unanswered. It also does not prevent the calls to `std::terminate` when one `noexcept` function is called in another:

```
bool lib::compute(int i) noexcept [[pre: i > 0]];

bool fun(int i) noexcept
{
    if (i < 0) lib::compute(-i);
    else      lib::compute(i); // bug on `i == 0`
}
```

Here, if we call `fun(0)` we violate the precondition of `lib::compute` which triggers an exception in some mode. Even if an exception is thrown outside of `lib::compute`, it is still called inside `fun`, which is also `noexcept`.

Exceptions thrown from predicates

Another source of exceptions caused by the contract support framework, unrelated to the violation handler, it when the evaluation of the predicate throws one. [\[P2388R4\]](#) requires that this should end in calling `std::terminate()`. SG21 had a consensus in Kona 2023 meeting that it was undesired, however no single alternative had consensus.

This paper does not address this problem explicitly, however, a lot of discussion related to throwing contract violation handlers also applies to throwing predicates.

Some real life experience

This section describes the author's experience with using throwing violation handlers.

In our production servers we use our own macro `IC_ASSERT`, which is evaluated in any "mode". In test builds contract violation ends in `std::abort()`, in order to collect a core dump and easily analyze the problem. In production builds we throw a special exception. This exception is only called in the main server loop: the client transaction is aborted, the client is responded with an apology message, and the server proceeds to processing the next transaction, which is hopefully sufficiently different that it does not enter the same path (maybe a new plugin) as the previous one. Often — although there is no guarantee — the stack unwinding is enough to recover from the bug.

This works satisfactorily due to a number of circumstances:

1. Macro `IC_ASSERT` is only used in the server code. We know it isn't used in the libraries we use. So, we control where it is put.
2. We do not annotate functions as `noexcept`. Practically all the functions provide only basic failure-safety guarantee. If anything goes wrong we just cancel either the entire client transaction, or skip a well isolated components.
3. We can afford to crash the server. Other sibling servers then take over the traffic, so even if we accidentally throw from a destructor due to a violated contract, it is tolerable.

Conclusion

In order to allow throwing from a contract runtime checks, we need to provide a clear semantics for all static checks that rely on the `noexcept` property: type traits, `noexcept`-operator, function types. For this we need a cleaner story for what `noexcept` specifier is. The model in this paper makes a distinction between a declaration of will for controlling overloads and a no-fail guarantee dependent on the precondition satisfaction. Only the former requires a syntactic marker to work, and conflating the two may not be beneficial. This takes us close to [N3248], except that our motivation is not to enable library validation.

It is also possible to disallow throwing from a violation handler (for instance, allow installing only `noexcept` contract violation handlers), if not at all, then only temporarily for the C++26 time frame. This would allow for violation handlers, but defer the specification of interactions with `noexcept` for the subsequent C++ release.

We also note that exceptions are not a good way to address the problem of applications that do want to stop in the face of the detected contract violation. They can be a cause for premature a termination on their own. In fact, a contract checking that can throw means that exceptions can now be thrown from any place, also functions without preconditions, which means that putting `noexcept` anywhere would be risky.

Acknowledgments

Joshua Berne has reviewed this document, and provided useful feedback.

References

- [N2855] — Douglas Gregor, David Abrahams, "Rvalue References and Exception Safety" (<https://www.open-std.org/JTC1/SC22/WG21/docs/papers/2009/n2855.html>).
- [N2983] — David Abrahams, Rani Sharoni, Doug Gregor, "Allowing Move Constructors to Throw" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2009/n2983.html>).
- [N3248] — Alisdair Meredith, John Lakos, "noexcept Prevents Library Validation" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2011/n3248.pdf>).
- [P0542R5] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0542r5.html>).
- [P2388R4] — Andrzej Krzemiński, Gašper Ažman, "Minimum Contract Support: either `No_eval` or `Eval_and_abort`" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2388r4.html>).
- [P2698R0] — Bjarne Stroustrup, "Unconditional termination is a serious problem" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2698r0.pdf>).
- [P2784R0] — Andrzej Krzemiński, "Not halting the program after detected contract violation" (<https://isocpp.org/files/papers/P2784R0.html>).
- [P2811R2] — Joshua Berne, "Contract-Violation Handlers" (<https://isocpp.org/files/papers/P2811R2.pdf>).
- [P2780R0] — Ville Voutilainen, "Caller-side precondition checking, and `Eval_and_throw`" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2780r0.html>).
- [P2852R0] — Tom Honermann, "Contract violation handling semantics for the contracts MVP" (<https://isocpp.org/files/papers/P2852R0.html>).